

บทที่ 5

Session Bean

session bean เป็น enterprise bean ตัวแทนงานที่ประมวลผลให้กับไคลเอนต์หนึ่ง ๆ มีจุดประสงค์เพื่อให้เป็นตัวแทนของบิซิเนสโพรเซส อาจเป็นงานเกี่ยวข้องกับลอจิก , อัลกอริทึม , หรือเวิร์คโฟลว์ ตัวอย่างของบิซิเนสโพรเซส เช่น การตรวจสอบบัตรเครดิต การรับรายการสั่งซื้อสินค้า การคำนวณต่าง ๆ หรือการติดตามยอดสินค้าคงคลัง บิซิเนสโพรเซสเหล่านี้สามารถแทนได้ด้วย session bean

5.1 ลักษณะของ Session Bean

5.1.1 ช่วงชีวิตของ Session Bean (Session Bean Lifetime)

ข้อแตกต่างหลักระหว่าง session bean กับ entity bean คือช่วงชีวิตของมัน session bean เป็นคอมโพเนนต์ที่มีช่วงชีวิตสั้น โดยมีช่วงชีวิตเท่ากับ session ของไคลเอนต์เท่านั้น เป็นที่มาของชื่อ session bean ระยะเวลาของเซสชันของไคลเอนต์นั้น อาจยาวนานเท่ากับระยะเวลาที่เว็บเบราว์เซอร์เปิดอยู่ , ระยะเวลาที่ Applet ทำงาน , ระยะเวลาที่แอปพลิเคชันแบบ stand-alone กำลังทำงาน หรือระยะเวลาที่ bean อื่น ๆ เข้ามาเรียกใช้ bean นั้นก็ได้

โดยทั่วไประยะเวลาของ session ของไคลเอนต์จะเป็นตัวกำหนดช่วงชีวิตของ session bean โดย EJB คอนเทนเนอร์จะทำลาย session bean เมื่อไคลเอนต์จบการทำงานไป ถ้าหากแอปพลิเคชันเซิร์ฟเวอร์หรือเครื่องคอมพิวเตอร์ที่ session bean อยู่เกิดแครช session bean จะถูกทำลายไปด้วย เพราะ session bean เป็นออบเจกต์ที่อยู่ในหน่วยความจำ ต้องอาศัยสภาวะแวดล้อมในการทำงาน

ในทางกลับกัน entity bean มีช่วงชีวิตยาวนาน เนื่องจากเป็นออบเจกต์ที่ persistent ซึ่งถือเป็นส่วนหนึ่งของแหล่งบันทึกข้อมูลถาวร เช่น ฐานข้อมูล โดย entity bean ถูกสร้างขึ้นในหน่วยความจำจากข้อมูลในฐานข้อมูล

session bean ไม่มีคุณลักษณะ persistent ไม่มีการบันทึกลงในแหล่งเก็บข้อมูลเหมือนกับ entity bean สามารถทำงานเกี่ยวกับฐานข้อมูลได้ แต่ตัว session bean เองไม่ใช่ออบเจกต์ที่ persistent

5.1.2 Session Bean ที่มีการเก็บสถานะและไม่มีการเก็บสถานะ

session bean มี 2 ชนิดคือ stateless session bean และ stateful session bean ข้อแตกต่างระหว่าง bean สองชนิดนี้คือ stateless session bean จะไม่มีการเก็บข้อมูลสถานะของไคลเอนต์ไว้ ในขณะที่ stateful session bean จะมีการเก็บสถานะของไคลเอนต์ไว้ เช่น ในระบบอี-คอมเมิร์ซ รถเข็นแบบออนไลน์จะต้องเก็บสถานะของสินค้าที่สั่งไว้ เป็นต้น

5.1.3 เมธอดทั้งหมดของ Session Bean จะต้องถูก Serialize

เมื่อเรียกเมธอดกับอินสแตนซ์ของ session bean EJB คอนเทนเนอร์จะรับประกันว่าจะไม่มีไคลเอนต์อื่นใช้อินสแตนซ์นั้น คอนเทนเนอร์จะป้องกันอินสแตนซ์ของ bean นั้นไว้และให้ไคลเอนต์ที่ทำงานอยู่พร้อม ๆ กันไปใช้อินสแตนซ์อื่นหรือคอยจนกว่าอินสแตนซ์ที่มีการใช้งานจะทำเสร็จแล้ว และถ้าหลาย ๆ ไคลเอนต์มีการเรียกใช้เมธอดกับ session bean พร้อม ๆ กัน การเรียกนั้นจะถูก serialize หรือทำใน lock-step ซึ่งหมายความว่าคอนเทนเนอร์จะจัดการลำดับการเรียกให้โดยอัตโนมัติ ทำให้อินสแตนซ์หนึ่งจะถูกใช้งานโดยไคลเอนต์เดียวเท่านั้น และเนื่องจากการร้องขอของไคลเอนต์จะถูก serialize ทำให้เราไม่ต้องเขียนโค้ดให้ bean มีคุณสมบัติ thread-safe จะมีเพียงเทรดเดียวของไคลเอนต์ที่จะทำงานกับ bean ในเวลาหนึ่ง ๆ

5.2 Stateless Session Bean

5.2.1 ลักษณะของ Stateless Session Bean

5.2.1.1 ไม่มีการเก็บสถานะ

stateless session bean ไม่เก็บสถานะการทำงานกับไคลเอนต์ใด ๆ ไว้ แม้จะสามารถเก็บสถานะภายในได้ แต่ไม่ได้เป็นสถานะเฉพาะสำหรับไคลเอนต์ตัวใดตัวหนึ่ง ในมุมมองของไคลเอนต์ stateless session bean ทุกตัวเหมือนกันหมด โดยไคลเอนต์ไม่สามารถแยกความแตกต่างได้ ในการใช้งานไคลเอนต์จะต้องส่งข้อมูลของไคลเอนต์ทั้งหมดที่ bean ต้องใช้ในการทำงานไปเป็นพารามิเตอร์ให้กับบิซิเนสเมธอด หรือ bean อาจดึงข้อมูลที่จำเป็นมาจากแหล่งภายนอก เช่นฐานข้อมูลก็ได้

5.2.1.2 สามารถกำหนดค่าเริ่มต้นให้กับ Stateless Session Bean ได้เพียงวิธีเดียว

session bean ถูกสร้างและกำหนดค่าเริ่มต้นจากเมธอด ejbCreate() แต่เนื่องจาก stateless session bean ไม่คงค่าสถานะเอาไว้ในการเรียกเมธอดแต่ละครั้ง ดังนั้นจึงไม่สามารถคงค่าใด ๆ ที่ไคลเอนต์ส่งเป็นพารามิเตอร์ให้กับเมธอด ejbCreate() ได้ stateless session bean ไม่จำเป็นต้องรองรับการเรียกใช้เมธอด ejbCreate() ได้หลายรูปแบบ เพราะในการเรียกใช้อินสแตนซ์ของ bean ครั้งต่อ ๆ ไป bean จะไม่มีข้อมูลของการเรียกใช้เมธอด ejbCreate() ในครั้งก่อนหน้าอยู่แล้ว

จากเหตุผลดังกล่าว stateless session bean จึงมีเมธอด ejbCreate() ได้เพียงตัวเดียว ที่ไม่รับค่าพารามิเตอร์ใด ๆ และใน home object จะมีเพียงเมธอด create() เข้าคู่กันซึ่งไม่รับพารามิเตอร์ใด ๆ เช่นกัน

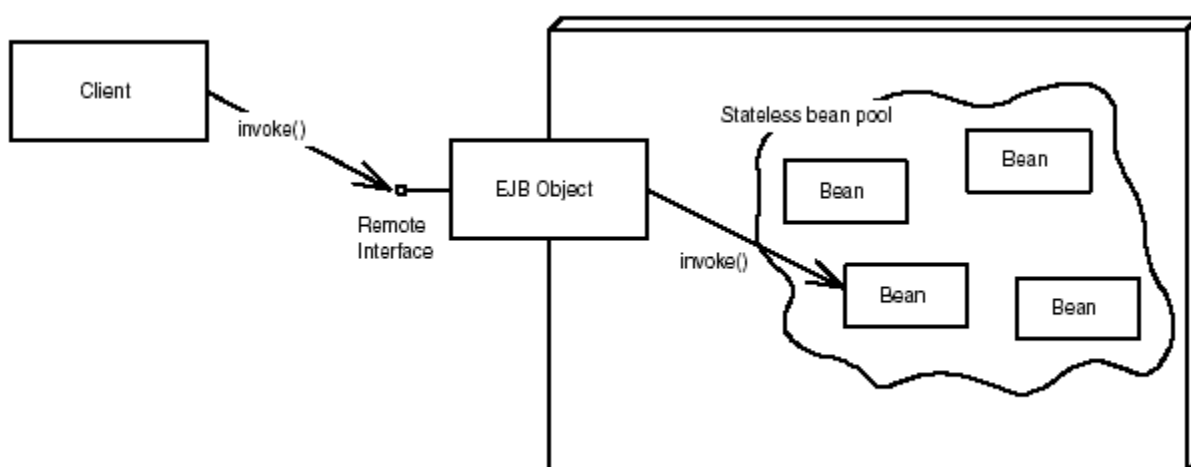
5.2.1.3 คอนเทนเนอร์สามารถพูล และนำ Stateless Session Bean กลับมาใช้ใหม่ได้

เนื่องจากเมธอด ejbCreate() ของ stateless session bean ไม่รับพารามิเตอร์ใด ๆ ไคลเอนต์จึงไม่มีการส่งข้อมูลเพื่อเริ่มต้นการทำงานของอินสแตนซ์ของ bean ดังนั้นคอนเทนเนอร์สามารถสร้างอินสแตนซ์ของ stateless session bean ไว้ก่อนที่ไคลเอนต์จะติดต่อเข้ามาได้ เมื่อไคลเอนต์เรียกใช้งานเมธอดคอนเทนเนอร์ก็ดึงอินสแตนซ์จากในพูลไปให้บริการกับไคลเอนต์ แล้วนำอินสแตนซ์นั้นกลับเข้าไปเก็บไว้

ในพูลเช่นเดิมเมื่อไคลเอนต์เลิกใช้งาน ทำให้คอนเทนเนอร์สามารถนำอินสแตนซ์ของ bean มาให้บริการกับไคลเอนต์ได้อย่างทันที

ผลที่ได้คือในแต่ละครั้งที่มีการร้องขอเข้ามาจากไคลเอนต์ อินสแตนซ์ของ session bean ใด ๆ สามารถให้บริการก็ได้ เนื่องจาก stateless session bean เก็บสถานะการทำงานระหว่างการเรียกใช้งานเมธอดในแต่ละครั้งเท่านั้น และไม่เก็บสถานะไคลเอนต์เอาไว้อีกหลังจากการเรียกใช้งานเมธอดเสร็จสิ้น stateless session bean แต่ละตัวจะอยู่ในสถานะเดียวกันเสมอหลังจากการเรียกใช้งานเมธอดในแต่ละครั้ง ดังนั้น คอนเทนเนอร์สามารถนำ stateless session bean ใด ๆ ไปให้บริการกับไคลเอนต์ได้ในทุก ๆ การเรียกใช้เมธอดแต่ละครั้ง stateless session bean หลาย ๆ ตัวสามารถให้บริการการเรียกใช้เมธอดแต่ละครั้งจากไคลเอนต์เดียวกันได้ ซึ่งการอิมพลิเมนต์วิธีการเลือก stateless session bean ให้กับไคลเอนต์จะแตกต่างกันไปตามคอนเทนเนอร์แต่ละผลิตภัณฑ์

ประโยชน์ของการพูลอินสแตนซ์คือ พูลของ bean สามารถมีจำนวนน้อยกว่าจำนวนไคลเอนต์ที่ติดต่อเข้ามาได้มาก เนื่องจากเวลาที่ใช้ในการคิดของไคลเอนต์ อาจเป็นเวลาที่ใช้ไประหว่างส่งข้อมูลผ่านเครือข่ายหรือเวลาที่มนุษย์ใช้ในการตัดสินใจบนฝั่งไคลเอนต์ ในระหว่างนั้นคอนเทนเนอร์อาจนำอินสแตนซ์ของ bean ไปให้บริการกับไคลเอนต์ตัวอื่นได้เพื่อประหยัดทรัพยากรของระบบ



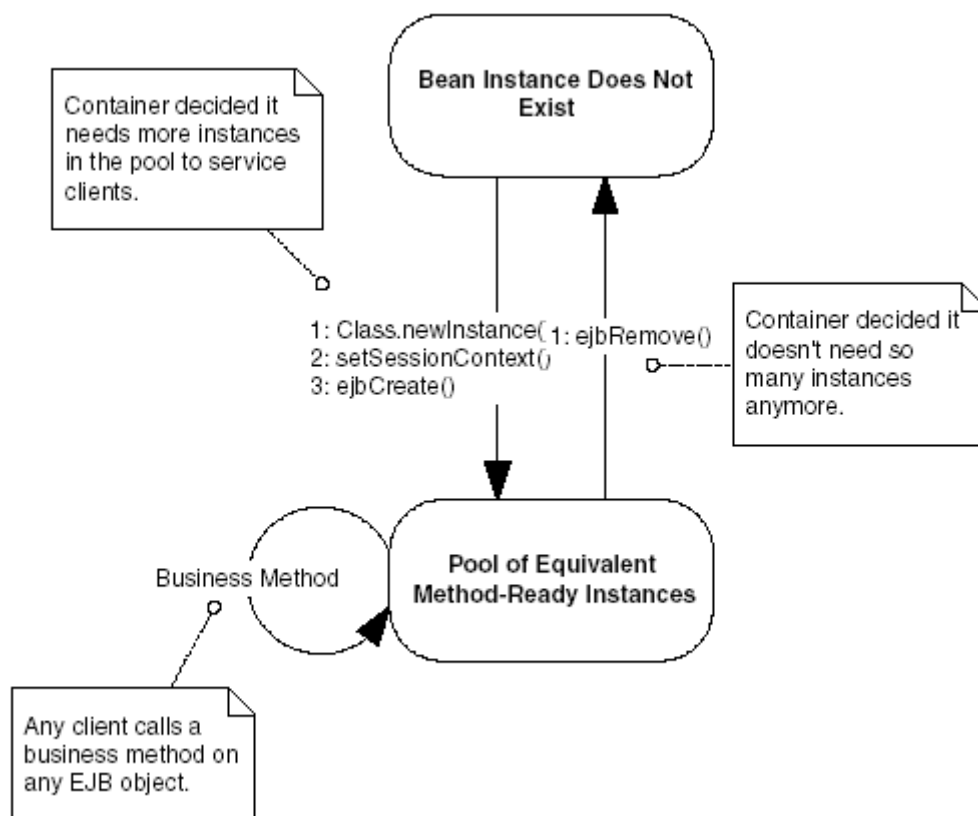
รูปที่ 5-1 การพูล Stateless Session Bean

ขนาดพูลของ bean ไม่จำเป็นต้องกำหนดไว้ตายตัว คอนเทนเนอร์ที่มีความสามารถสูงสามารถปรับขนาดของพูลได้อย่างไดนามิก โดยอัตโนมัติขณะโหลดเปลี่ยนแปลง เช่น ถ้าหากมีไคลเอนต์เข้ามาใช้งานตอนกลางวันมากกว่าตอนกลางคืน คอนเทนเนอร์อาจจะสร้างพูลขนาดใหญ่ในตอนกลางวัน และขนาดเล็กกว่าในตอนกลางคืน ช่วยให้ระบบมีทรัพยากรเหลือมากขึ้นสำหรับทำงานอย่างอื่นในช่วงเวลาที่ระบบมีโหลดไม่มาก

การพูลอินสแตนซ์ของ stateless session bean แสดงในรูปที่ 5-1

5.2.1.4 วงจรชีวิตของ Stateless Session Bean

วงจรชีวิตของ stateless session bean ดังรูปที่ 5-2



รูปที่ 5-2 วงจรชีวิตของ *Stateless Session Bean*

5.2.2 ตัวอย่างโค้ด Stateless Session Bean

ตัวอย่างของ stateless session bean นี้เป็น bean ที่ใช้ในการแปลงค่าเงิน โดยในการโค้ดจะต้องประกอบด้วยส่วนต่าง ๆ ดังนี้

5.2.2.1 Remote Interface

remote interface จะต้องนิยามเมธอดทั้งหมดที่มีอยู่ใน bean คอนเทนเนอร์จะอิมพลีเมนต์ remote interface อิมพลีเมนต์นั้นคือ EJB object และ EJB object จะส่ง request ของไคลเอนต์ต่อไปยัง bean ที่แท้จริง โค้ดจะเป็นดังนี้

Converter.java

```
package olala;
import javax.ejb.*;
```

```
import java.rmi.RemoteException;

public interface Converter extends EJBObject{

    public double yenToEuro(double yen)    throws RemoteException;

    public double dollarToYen(double dollars)    throws RemoteException;

}
```

remote interface จะต้องขยายมาจาก javax.ejb.EJBObject ซึ่งหมายความว่าคอนเทนเนอร์จะสร้าง EJB object ให้ โดยจะอิมพลีเมนต์จาก remote interface และจะมีทุกเมธอดที่มีใน remote interface โดยตัวอย่างนี้มีบิซิเนสเมธอดคือ yenToEuro(double yen) และ dollarToYen(double dollars) โดยจะต้องอิมพลีเมนต์เมธอดนี้ใน enterprise bean class และเนื่องจาก remote interface ขยายมาจาก java.rmi.Remote มันจะโยน remote exception ด้วย ซึ่งเป็นข้อแตกต่างของบิซิเนสเมธอดในอินเทอร์เฟซ และ enterprise bean class

5.2.2.2 Enterprise Bean Class

bean จะต้องขยายมาจากอินเทอร์เฟซ javax.ejb.SessionBean โค้ดจะเป็นดังนี้

ConverterEJB.java

```
package olala;

import javax.ejb.*;
import java.util.*;
import javax.naming.*;

public class ConverterEJB implements javax.ejb.SessionBean{

    private SessionContext ctx;

    public ConverterEJB() {}

    public void ejbCreate() {}

    public void ejbActivate() {}

    public void ejbPassivate() {}

    public void ejbRemove() {}

    public void setSessionContext(SessionContext ctx) { this.ctx = ctx; }

    public double yenToEuro(double yen) { return yen*0.0077; }

    public double dollarToYen(double dollars) { return dollars*121.6000; }

}
```

5.2.2.3 Home Interface

home interface จะใช้ในการระบุกลไกในการสร้างและทำลาย EJB object โค้ดดังนี้

ConverterHome.java

```
package olala;
import javax.ejb.*;
import java.rmi.RemoteException;
public interface ConverterHome extends EJBHome{
    public Converter create() throws CreateException, RemoteException;
}
```

home interface จะต้องขยายมาจาก javax.ejb.EJBHome EJBHome จะกำหนดวิธีในการทำลาย EJB object เอง จึงไม่ต้องเขียนเมธอดนี้ home interface จะต้องมีเมธอดที่ใช้สร้าง EJB object โดยไม่ต้องมีพารามิเตอร์ใด ๆ และเมธอด create() จะต้องโยน java.rmi.RemoteException และ javax.ejb.CreateException

5.2.2.4 Deployment Descriptor

จะต้องประกอบด้วย ejb-xml.jar และ weblogic-ejb-xml.jar โดยภายใน descriptor สามารถมีรายละเอียดของ bean ได้มากกว่า 1 bean

ejb-xml.jar

```
<?xml version="1.0"?>
<!DOCTYPE ejb-jar PUBLIC "-//Sun Microsystems, Inc.//DTD Enterprise JavaBeans 1.1//EN"
'http://java.sun.com/j2ee/dtds/ejb-jar_1_1.dtd'>
<ejb-jar>
  <enterprise-beans>
    <session>
      <description></description>
      <ejb-name>Converter</ejb-name>
      <home>olala.ConverterHome</home>
      <remote>olala.Converter</remote>
      <ejb-class>olala.ConverterEJB</ejb-class>
```

```

    <session-type>Stateless</session-type>

    <transaction-type>Container</transaction-type>

</session>

</enterprise-beans>

<assembly-descriptor>

    <container-transaction>

        <method>

            <ejb-name>Converter</ejb-name>

            <method-name>*</method-name>

        </method>

        <trans-attribute>Required</trans-attribute>

    </container-transaction>

</assembly-descriptor>

</ejb-jar>

```

weblogic-ejb-jar.xml

```

<?xml version="1.0"?>

<!DOCTYPE weblogic-ejb-jar PUBLIC "-//BEA Systems, Inc.//DTD WebLogic 5.1.0 EJB//EN"
'http://www.bea.com/servers/wls510/dtd/weblogic-ejb-jar.dtd'>

<weblogic-ejb-jar>

    <weblogic-enterprise-bean>

        <ejb-name>Converter</ejb-name>

        <jndi-name>Converter</jndi-name>

    </weblogic-enterprise-bean>

</weblogic-ejb-jar>

```

5.2.2.5 คอมไพล์และแพ็คเกจ

คอมไพล์จาวาไฟล์ จัดไฟล์ทั้งหมดอยู่ใน path ดังนี้

./Converter/olala/Converter.class

./Converter/olala/ConverterEJB.class

./Converter/olala/ConverterHome.class

./Converter/meta-inf/ejb-jar.xml

./Converter/meta-inf/weblogic-ejb-jar.xml

สร้าง jar ไฟล์โดยใช้คำสั่งนี้

```
./Converter>jar cv0f stdConverter.jar olala meta-inf
```

และทำคำสั่งต่อไปนี้

```
./Converter/java weblogic.ejbrc -compiler javac stdConverter.jar Converter.jar
```

แล้วจึงนำ Converter.jar ไปดีพลอยโดยดูวิธีการติดตั้ง WebLogic Server และวิธีดีพลอยได้ในภาคผนวก ก

5.2.2.6 ไคลเอ็นต์

เซต classpath ที่ไปยัง Converter.jar หรือ path ที่มีอินเทอร์เฟซของ bean โค้ดไคลเอ็นต์ดังนี้

ConverterClient.java

```
import olala.*;
import javax.ejb.*;
import javax.naming.*;
import java.util.*;

public class ConverterClient{
public static void main(String[] args) {
    try {
        Context ctx = getInitialContext();
        ConverterHome home = (ConverterHome) ctx.lookup("Converter");
        Converter the_ejb = home.create();
        double euro=the_ejb.yenToEuro(100);
        System.out.println("100 yen = "+euro+" euro");
        double yen=the_ejb.dollarToYen(100);
        System.out.println("100 dollar = "+yen+" yen");
        the_ejb.remove();
    } catch (Exception e) {
        e.printStackTrace();
    }
}

public static Context getInitialContext() throws NamingException {
    Properties p = new Properties();
    p.put(Context.INITIAL_CONTEXT_FACTORY, "weblogic.jndi.WLInitialContextFactory");
    p.put(Context.PROVIDER_URL, url);
}
```



```

if (user != null) {
    p.put(Context.SECURITY_PRINCIPAL, user);
    if (password == null) password = "";
    p.put(Context.SECURITY_CREDENTIALS, password);
}

return new InitialContext(p);
}

static String url = "t3://localhost:7001";

static String user = null;

static String password = null;
}

```

คอมไพล์แล้วรันจะได้ผลดังนี้

```

100 yen = 0.77 euro
100 dollar = 12160.0 yen

```

5.3 Stateful Session Bean

stateful session bean เป็น bean ที่เก็บสถานะการทำงานกับไคลเอนต์หนึ่ง ๆ ครอบคลุมระยะเวลาการเรียกเมธอดหลายครั้ง

5.3.1 ลักษณะของ Stateful Session Bean

5.3.1.1 การพุด Stateful Session Bean

ในสถานการณ์ที่ไคลเอนต์จำนวนมากกำลังทำงานกับ stateful session bean จำนวนหนึ่งในคอนเทนเนอร์ โดยปกติไคลเอนต์ต้องมีระยะเวลาในการคิด หรืออยู่ในสถานะที่ห่างไกลออกไปมาก และระบบเครือข่ายสามารถส่งข้อมูลได้ช้า แต่หมายความว่าต้องมี stateful session bean จำนวนเท่า ๆ กันกับไคลเอนต์ในคอนเทนเนอร์ โดย stateful session bean แต่ละตัวจะเก็บสถานะการทำงานของไคลเอนต์แต่ละตัวเอาไว้ แต่คอนเทนเนอร์มีทรัพยากรที่จำกัด เช่น หน่วยความจำ ช่องทางติดต่อกับฐานข้อมูล (Socket Connection) หากว่าสถานะของการทำงานที่เก็บนั้นมีขนาดใหญ่ คอนเทนเนอร์ก็จะมีทรัพยากรเหลือไม่เพียงพอแก่การทำงานได้ ซึ่งปัญหานี้จะไม่เกิดกับ stateless session bean เพราะคอนเทนเนอร์สามารถพุด bean จำนวนน้อยเพื่อให้บริการกับไคลเอนต์จำนวนมากได้

การพุด stateful session bean ทำได้ไม่ง่าย เมื่อไคลเอนต์ติดต่อเข้ามายัง bean ไคลเอนต์นั้นจะเริ่มการทำงานกับ bean ทันที และสถานะการทำงานที่เก็บอยู่ใน bean จะต้องคงอยู่ในการเรียกใช้เมธอดครั้ง

ต่อไปของไคลเอ็นต์ตัวเดิมนั้น ดังนั้นคอนเทนเนอร์จะไม่สามารถพูล bean ขึ้นมาและนำไปให้บริการการเรียกใช้งานเมธอดต่าง ๆ แบบไดนามิกได้ เนื่องจาก bean แต่ละตัวนั้น เก็บสถานะโดยจะจกกับไคลเอ็นต์แต่ละตัวเอาไว้

ปัญหาดังกล่าว ใกล้เคียงกับปัญหาที่เกิดขึ้นในระบบปฏิบัติการ เมื่อเรียกใช้แอปพลิเคชันบนเครื่องคอมพิวเตอร์ จะมีหน่วยความจำแบบฟิสิคอลลที่จำกัดสำหรับให้โปรแกรมทำงาน ระบบปฏิบัติการต้องทำให้แอปพลิเคชันทำงานอยู่ได้ แม้ว่าแอปพลิเคชันจะต้องการปริมาณหน่วยความจำมากกว่าหน่วยความจำจริง ๆ ทั้งหมดที่มีอยู่ โดยระบบปฏิบัติการจะใช้นี้ในฮาร์ดดิสก์เป็นส่วนเพิ่มขยายของหน่วยความจำเป็นการเพิ่มหน่วยความจำเสมือน (virtual memory) ของระบบ เมื่อแอปพลิเคชันใด ๆ ที่ยังไม่มีการทำงาน ข้อมูลในหน่วยความจำที่มันใช้งานอยู่ก็จะถูกย้าย (swap) ออกจากหน่วยความจำลงไปยังฮาร์ดดิสก์ เมื่อแอปพลิเคชันนั้นมีการทำงานอีกครั้ง ข้อมูลที่จำเป็นก็จะถูกย้ายกลับเข้ามาในหน่วยความจำ การสลับข้อมูลไปมาจะเกิดขึ้นบ่อยครั้งเมื่อมีการสลับการทำงานไปมาระหว่างแอปพลิเคชัน (context switching)

EJB คอนเทนเนอร์ใช้วิธีแก้ปัญหที่เกิดขึ้นกับ stateful session bean ในลักษณะเดียวกัน เพื่อจำกัดจำนวนอินสแตนซ์ของ stateful session bean ในหน่วยความจำ คอนเทนเนอร์สามารถ swap stateful session bean ออกไปได้ โดยอาจเก็บสถานะของ bean ไว้ในฮาร์ดดิสก์หรือแหล่งเก็บข้อมูลอื่น การ swap stateful session bean ออกไปเรียกว่าการทำ passivation ทำให้ได้หน่วยความจำและทรัพยากรต่าง ๆ กลับคืนมา เมื่อไคลเอ็นต์ตัวเดิมเรียกใช้เมธอดของ bean อีกครั้ง สถานะการทำงานกับไคลเอ็นต์ที่ถูก passivate ออกไปจะถูก swap กลับเข้ามาใน stateful session bean อีกครั้ง วิธีการนี้เรียกว่า activation ทำให้ bean สามารถทำงานกับไคลเอ็นต์ต่อไปได้

bean ที่ได้รับสถานะโดยการ activate เข้ามาอาจจะไม่ใช่อินสแตนซ์ของ bean ตัวเดิมที่เคยติดต่อกับไคลเอ็นต์ตัวนั้นก็ได้ ซึ่งไม่ทำให้เกิดผลใด ๆ เนื่องจากอินสแตนซ์ของ bean ตัวใหม่นั้น จะทำงานต่อไปจากจุดเดิมที่สถานะของการทำงานได้ถูก passivate เอาไว้

ดังนั้นอินสแตนซ์จำนวนจำกัดเท่านั้นที่จะอยู่ในหน่วยความจำ ขณะที่ไคลเอ็นต์จำนวนมากมายติดต่อเข้ามา แต่การพูลวิธีนี้ก็ยังมีข้อเสีย เพราะการทำ passivation และ activation ต้องมีการใช้ I/O ด้วย อาจทำให้เกิดกรณีคอขวด (bottleneck) ขึ้นได้ ซึ่งแตกต่างจาก stateless session bean ซึ่งสามารถทำพูลได้ง่ายเนื่องจากไม่ต้องเก็บค่าสถานะการทำงานเอาไว้

การทำ passivation อาจเกิดขึ้นได้ทุกขณะ เมื่อ bean นั้นไม่ได้เกี่ยวข้องกับการเรียกใช้งานเมธอดในขณะนั้น การตัดสินใจว่าจะ passivate bean เมื่อไหร่ขึ้นอยู่กับคอนเทนเนอร์ โดยมีข้อยกเว้นอยู่ว่า bean ซึ่งทำงานอยู่ในทรานแซกชัน จะไม่สามารถ passivate ได้จนกว่าทรานแซกชันนั้นจะเสร็จสมบูรณ์

ส่วนการ activation bean จะเกิดขึ้นเมื่อ bean นั้นได้รับการร้องขอเข้ามา หากไคลเอ็นต์ใด ๆ ทำการร้องขอเข้ามา แต่สถานะการทำงานของมันถูก passivate อยู่ คอนเทนเนอร์จะ activate bean และอ่านข้อมูลที่ถูก passivate อยู่กลับเข้ามาในหน่วยความจำ

โดยทั่วไป การทำ passivation และ activation ไม่มีประโยชน์สำหรับ stateless session bean เนื่องจากไม่มีสถานะที่จะทำการ passivate และ activate และการทำ passivation และ activation นั้นจะถูกนำไปใช้กับ entity bean ด้วย จะได้กล่าวถึงต่อไป

5.3.1.2 ข้อบังคับในการเก็บสถานะของไคลเอนต์

สถานะการทำงานของ bean เป็นไปตามกฎที่วางไว้โดย java object Serialization เมื่อคอนเทนเนอร์ passivate bean จะใช้ object serialization ในการแปลงสถานะการทำงานของ bean ไปเป็นบิตข้อมูล หลังจากนั้นจึงเขียนบิตข้อมูลเหล่านั้นลงไปยังแหล่งเก็บข้อมูล เมื่อ bean ถูกเขียนลงไปยังแหล่งเก็บข้อมูลแล้ว หน่วยความจำที่มันใช้อยู่ก็จะถูกปล่อยโดย garbage collector ได้ ส่วนการ activate ก็เป็นขั้นตอนที่ตรงข้ามกัน โดยบิตข้อมูลที่ถูก serialize ที่เก็บอยู่ในแหล่งเก็บข้อมูลจะถูกอ่านกลับขึ้นมาในหน่วยความจำและแปลงไปเป็นข้อมูลของ bean ซึ่งอยู่ในหน่วยความจำ สิ่งที่ทำให้วิธีการเหล่านี้สามารถทำได้ คือการที่อินเทอร์เฟซ javax.ejb.EnterpriseBean สืบทอดมาจาก java.io.Serializable และทุก ๆ enterprise bean อิมพลีเมนต์อินเทอร์เฟซนี้

สำหรับจาวาออบเจกต์ที่เป็นส่วนหนึ่งของสถานะของ bean ก็ใช้หลักการเดียวกันคือออบเจกต์นั้นจะต้องอิมพลีเมนต์ java.io.Serializable ด้วย แม้ว่า bean จะต้องเป็นไปตามกฎของ object serialization แต่ EJB คอนเทนเนอร์ไม่จำเป็นต้องใช้โพรโตคอลมาตรฐานในการทำ serialization เสมอไป อาจใช้โพรโตคอลของตัวเองซึ่งสามารถช่วยเพิ่มความยืดหยุ่นและความแตกต่างระหว่างคอนเทนเนอร์ของแต่ละผู้ผลิต

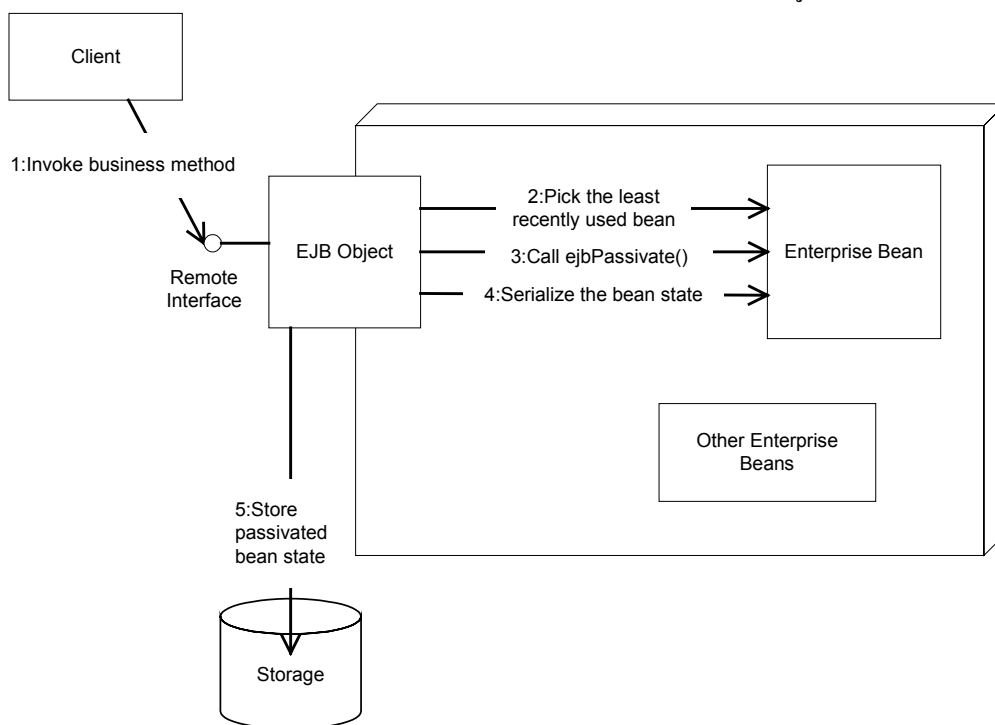
5.3.1.3 ขั้นตอนการทำ Passivation และ Activation

เมื่อ EJB คอนเทนเนอร์ passivate bean คอนเทนเนอร์จะเก็บสถานะการทำงานของ bean นั้นลงไปในแหล่งเก็บข้อมูล เช่นไฟล์ หรือฐานข้อมูล โดยคอนเทนเนอร์จะบอกให้ bean ทราบว่า มันกำลังจะทำการ passivate ด้วยการเรียกใช้เมธอด ejbPassivate() เมธอดนี้เป็นการเตือน bean ว่า สถานะการทำงานของมันกำลังจะถูก swap ออกไป

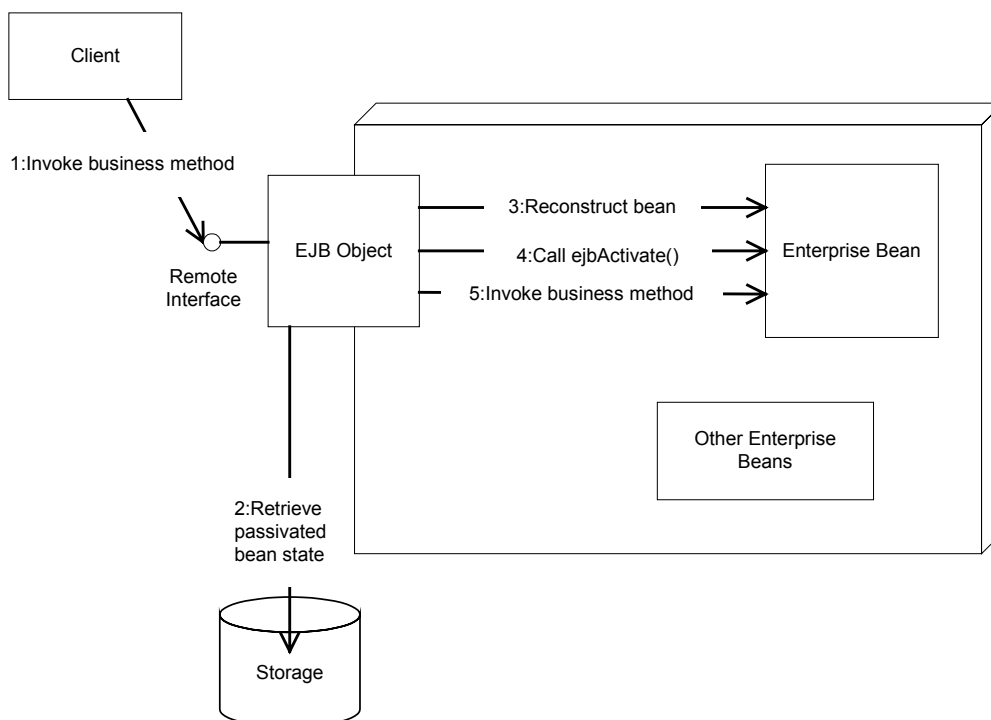
การที่คอนเทนเนอร์บอกให้ bean ทราบด้วยการเรียกใช้เมธอด ejbPassivate() นั้นเป็นสิ่งจำเป็น ทำให้ bean สามารถปล่อยทรัพยากรที่มันกำลังถือครองอยู่ได้ ทรัพยากรในที่นี้อาจเป็นช่องทางการติดต่อฐานข้อมูล , ซ็อกเก็ต , ไฟล์ที่เปิดอยู่ หรือทรัพยากรอื่น ๆ ซึ่งไม่สามารถเก็บลงในดิสก์ได้ หรือไม่มีความจำเป็นที่จะต้องเก็บไว้ เมื่อการเรียกเมธอด ejbPassivate() ของ bean เสร็จสมบูรณ์ bean ก็จะอยู่ในสถานะพร้อมถูก passivate การทำ passivation แสดงในรูปที่ 5-3

ขั้นตอนตรงกันข้ามจะเกิดขึ้นในกรณีของการ activate bean ซึ่งสถานะของการทำงานที่ถูก serialize จะถูกอ่านกลับเข้ามาในหน่วยความจำ และคอนเทนเนอร์จะสร้างสถานะขึ้นมาใหม่ในหน่วยความจำ โดยใช้ Object Serialization หรือสิ่งที่เหมือนกัน (equivalent) ในการแปลง หลังจากนั้นคอนเทนเนอร์จะเรียก

เมธอด `ejbActivate()` ทำให้ bean สามารถเรียกใช้ทรัพยากรต่าง ๆ ที่จำเป็น ซึ่งถูกปล่อยไปขณะถูก `passivate` ด้วยเมธอด `ejbPassivate()` กลับมา ขั้นตอนการทำ activation แสดงในรูปที่ 5-4



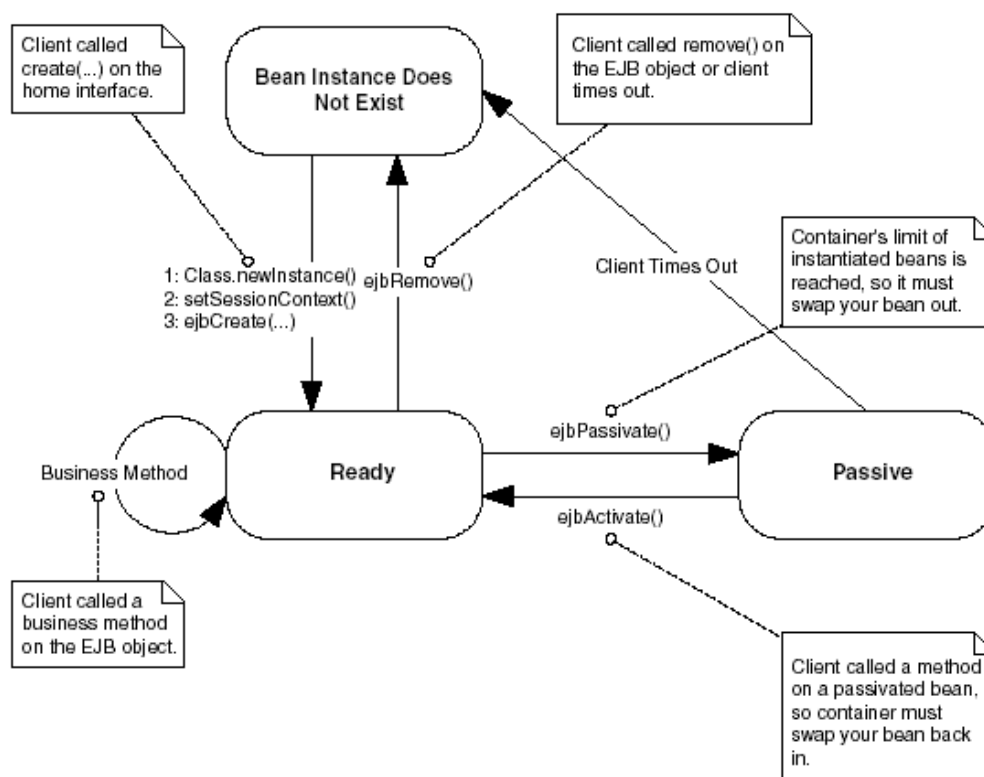
รูปที่ 5-3 การ *Passivate Stateful Session Bean*



รูปที่ 5-4 การ *Activate Stateful Session Bean*

5.3.1.4 วงจรชีวิตของ Stateful Session Bean

วงจรชีวิตของ stateful session bean ดังรูปที่ 5-5



รูปที่ 5-5 วงจรชีวิตของ Stateful Session Bean

5.3.2 ตัวอย่างโค้ด Stateful Session Bean

ตัวอย่างของ bean คือ bean ที่ใช้นับ

5.3.2.1 Remote Interface

remote interface จะมีเพียงเมธอดเดียวคือ count() ซึ่งจะต้องอิมพลิเมนต์ใน bean โค้ดดังนี้

Count.java

```

package olala;
import javax.ejb.*;
import java.rmi.RemoteException;
public interface Count extends EJBObject {
    public int count()
        throws RemoteException;
}
  
```

5.3.2.2 Enterprise Bean Class

โค้ดดังนี้

CountEJB.java

```
package olala;
import javax.ejb.*;
import java.util.*;
import javax.naming.*;
public class CountEJB implements javax.ejb.SessionBean{
    private SessionContext ctx;
    private int val;
    public CountEJB() {}
    public void ejbActivate() {}
    public void ejbPassivate() {}
    public void ejbRemove() {}
    public void setSessionContext(SessionContext ctx) { this.ctx = ctx; }
    public void ejbCreate(int val) { this.val=val; }
    public int count() { return ++val; }
}
```

bean จะต้องอิมพลีเมนต์ javax.ejb.SessionBean ซึ่งหมายความว่า bean ต้องมีทุกเมธอดที่มีในอินเทอร์เฟซ SessionBean เมธอด ejbCreate() จะมีพารามิเตอร์ที่ใช้ในการกำหนดค่าเริ่มต้นให้กับ bean ซึ่งแตกต่างจาก stateless ที่จะมีเมธอด ejbCreate() โดยไม่มีพารามิเตอร์เมธอดเดียวเท่านั้น

5.3.2.3 Home Interface

โค้ดดังนี้

CountHome.java

```
package olala;
import javax.ejb.*;
import java.rmi.RemoteException;
public interface CountHome extends EJBHome {
    public Count create(int val) throws CreateException, RemoteException;
}
```

5.3.2.4 Deployment Descriptor

จะประกอบด้วย descriptor 2 ไฟล์เหมือนกับ stateless session bean ดังนี้

ejb-jar.xml

```
<?xml version="1.0"?>
<!DOCTYPE ejb-jar PUBLIC "-//Sun Microsystems, Inc.//DTD Enterprise JavaBeans 1.1//EN"
'http://java.sun.com/j2ee/dtds/ejb-jar_1_1.dtd'>
<ejb-jar>
  <enterprise-beans>
    <session>
      <description></description>
      <ejb-name>Count</ejb-name>
      <home>olala.CountHome</home>
      <remote>olala.Count</remote>
      <ejb-class>olala.CountEJB</ejb-class>
      <session-type>Stateful</session-type>
      <transaction-type>Container</transaction-type>
    </session>
  </enterprise-beans>
  <assembly-descriptor>
    <container-transaction>
      <method>
        <ejb-name>Count</ejb-name>
        <method-name>*</method-name>
      </method>
      <trans-attribute>Required</trans-attribute>
    </container-transaction>
  </assembly-descriptor>
</ejb-jar>
```

weblogic-ejb-jar.xml

```
<?xml version="1.0"?>
<!DOCTYPE weblogic-ejb-jar PUBLIC "-//BEA Systems, Inc.//DTD WebLogic 5.1.0 EJB//EN"
'http://www.bea.com/servers/wls510/dtd/weblogic-ejb-jar.dtd'>
<weblogic-ejb-jar>
  <weblogic-enterprise-bean>
    <ejb-name>Count</ejb-name>
    <caching-descriptor>
      <max-beans-in-cache>5</max-beans-in-cache>
      <idle-timeout-seconds>10</idle-timeout-seconds>
    </caching-descriptor>
    <enable-call-by-reference>True</enable-call-by-reference>
    <jndi-name>Count</jndi-name>
  </weblogic-enterprise-bean>
</weblogic-ejb-jar>
```

5.3.2.5 คอมไพล์และแพ็คเกจ

คอมไพล์จาวาไฟล์ จัดไฟล์ทั้งหมดอยู่ใน path ดังนี้

./Count/olala/Count.class

./Count/olala/CountEJB.class

./Count/olala/CountHome.class

./Count/meta-inf/ejb-jar.xml

./Count/meta-inf/weblogic-ejb-jar.xml

สร้าง jar ไฟล์โดยใช้คำสั่งดังนี้

/Count>jar cv0f stdCount.jar olala meta-inf

และทำคำสั่งต่อไปนี้

./Count/java weblogic.ejbrc -compiler javac stdCount.jar Count.jar

แล้วจึงนำ Count.jar ไปดีพลอย

5.3.2.6 ไคลเอ็นต์

เซต classpath ที่ไปยัง Count.jar หรือ path ที่มีอินเทอร์เฟซของ bean โค้ดไคลเอ็นต์ดังนี้

CountClient.java

```
import olala.*;
import javax.ejb.*;
import javax.naming.*;
import java.util.*;

public class CountClient{

    public static void main(String[] args) {

        try {

            Context ctx = getInitialContext();

            CountHome home = (CountHome) ctx.lookup("Count");

            Count count[] = new Count[3];

            int countVal=0;

            System.out.println("Instantiating beans...");

            for (int i=0;i<3;i++) {

                count[i]=home.create(countVal);

                countVal=count[i].count();

                System.out.println(countVal);

                Thread.sleep(500);

            }

            System.out.println("Calling count() on beans...");

            for (int i=0;i<3;i++) {

                countVal=count[i].count();

                System.out.println(countVal);

                Thread.sleep(500);

            }

            for (int i=0;i<3;i++) { count[i].remove(); }

        }

        catch (Exception e) { e.printStackTrace(); }

    }

    public static Context getInitialContext() throws NamingException{
```

```

Properties p = new Properties();

p.put(Context.INITIAL_CONTEXT_FACTORY, "weblogic.jndi.WLInitialContextFactory");

p.put(Context.PROVIDER_URL, url);

if (user != null) {

    p.put(Context.SECURITY_PRINCIPAL, user);

    if (password == null)

        password = "";

    p.put(Context.SECURITY_CREDENTIALS, password);

}

return new InitialContext(p);

}

static String url = "t3://localhost:7001";

static String user = null;

static String password = null;

}

```

คอมไพล์แล้วรันจะได้ผลดังนี้

Instantiating beans...

1

2

3

Calling count() on beans...

2

3

4